EXPERIMENT NO.: 14

****TITLE: PRO-CODE EXTENSION OF A LOW-CODE APPLICATION*****

****Roll Number:** 241401087***
****Platform:** Zoho Creator and Zoho Catalyst***
****Application:** Loan Application and Pre-screening Management***

1. Aim

To understand how pro-code development and external services can extend the capabilities of a low-code application and support requirements that cannot be handled conveniently by built-in components alone.

2. Software Requirements

- Zoho Creator account
- Zoho Catalyst account, where required by the experiment
- Web browser and internet connection
- Loan Application and Pre-screening Management application

3. Theory

Pro-code development involves writing conventional program code to implement specialized business logic, algorithms, integrations, and processing tasks.

Low-code platforms simplify application creation through visual tools, whereas pro-code techniques provide more flexibility and control. Combining both approaches can help organizations develop applications quickly while supporting advanced requirements.

Zoho Catalyst provides cloud-based application development services that can support custom backend logic and other application extensions. A low-code application may communicate with an external service through an API or another supported integration method.

Benefits of pro-code extension

1. Supports specialized business logic.
2. Enables integration with external systems and APIs.
3. Provides greater control over complex processing.
4. Helps separate backend processing from the user interface.
5. Can support evolving application requirements.

Important considerations

Custom code requires testing, maintenance, error handling, authentication, and appropriate security controls. External integrations should not expose API credentials or confidential applicant information.

4. Case Study

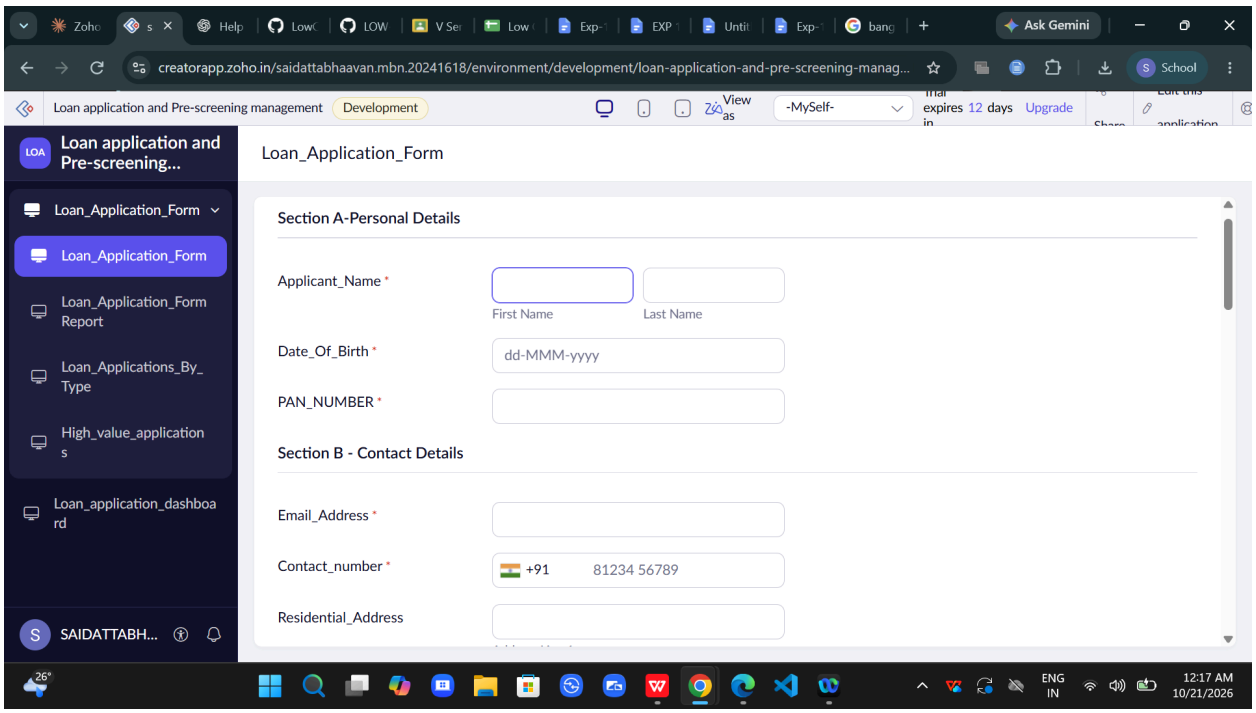
In the Loan Application and Pre-screening Management application, basic applicant data collection can be performed through Zoho Creator forms and reports.

A future extension could process loan-related calculations or communicate with an authorized external service. Such an extension would need defined inputs, validation, secure communication, error handling, and verified outputs.

5. Procedure

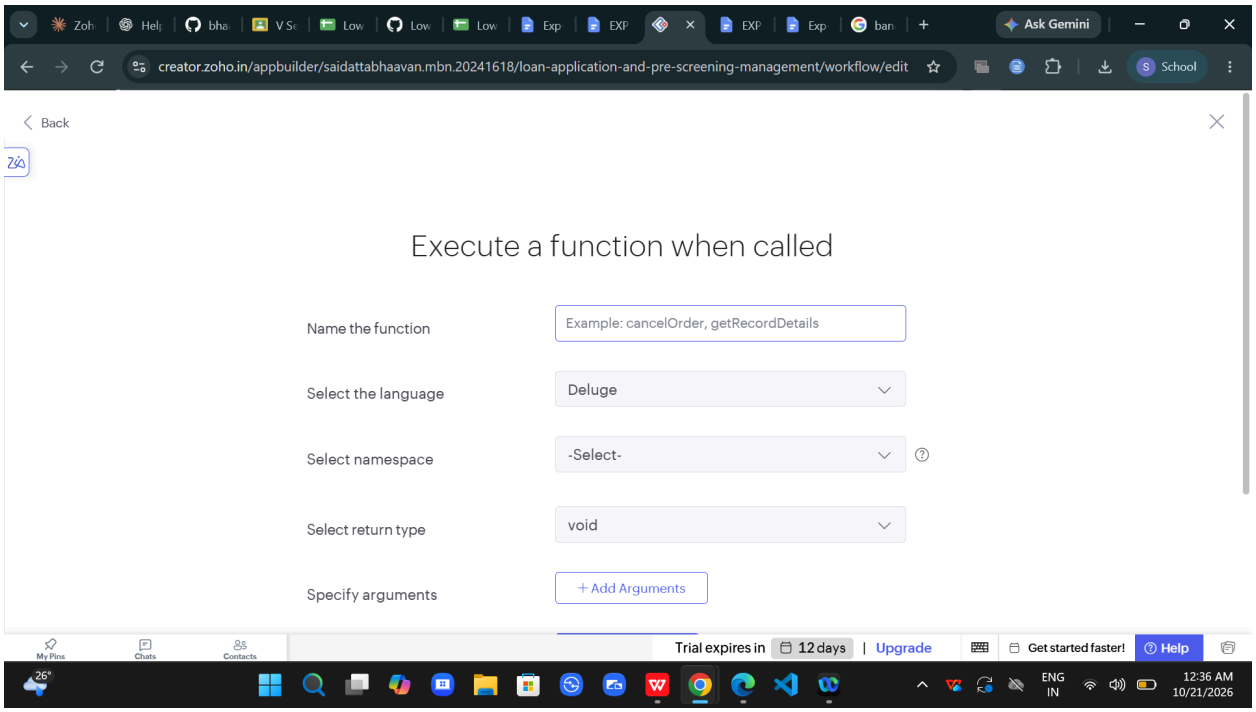
Step 1: Examine the low-code application

1. Log in to Zoho Creator.
2. Open the Loan Application and Pre-screening Management application.
3. Inspect the loan application form and its existing fields.
4. Identify one requirement that may benefit from custom programming.



Step 2: Identify the extension requirement

1. Define the task to be handled by custom code.
2. Identify the required input values and expected output.
3. Determine the validation and error-handling requirements.
4. Decide whether the task needs an external service or can be handled with an available platform function.



Step 3: Explore the pro-code platform

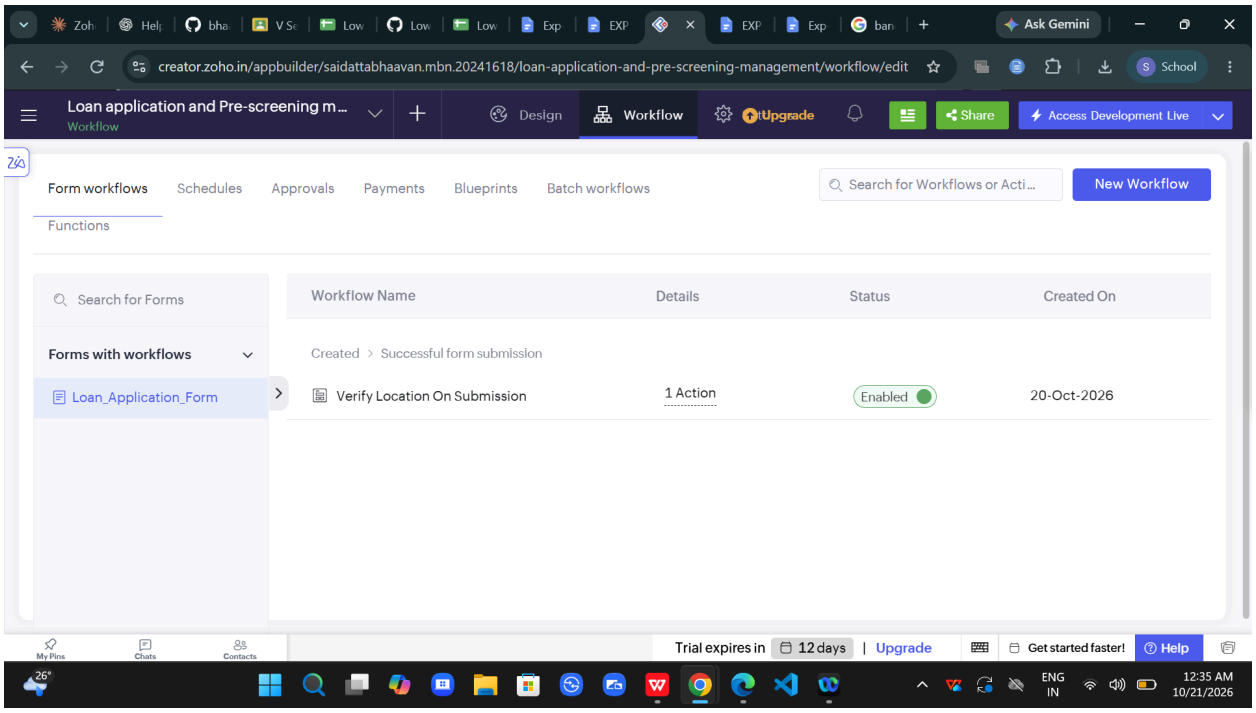
1. Open Zoho Catalyst if it is part of the required practical workflow.
2. Examine the available project or application development options.
3. Identify the relevant backend function or service supported by the manual.
4. Record the development environment and its purpose.

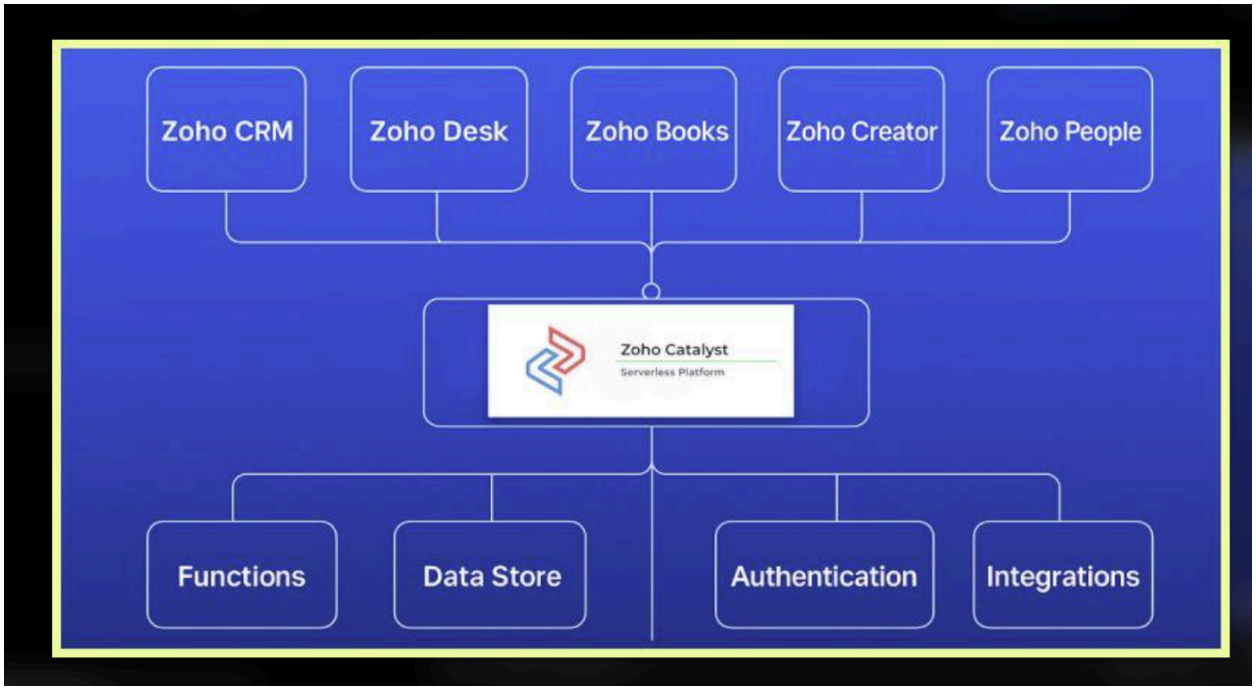
Step 4: Develop the extension

1. Follow the reference manual to create the specified function or component.
2. Implement the required logic using the supported programming environment.
3. Validate input values before processing.
4. Return a clearly defined output or error response.
5. Test the extension using non-sensitive sample data.

Step 5: Integrate and test

1. Follow the manual's prescribed integration method between the low-code application and the custom component.
2. Configure the required connection or API settings.
3. Test a valid input and check the returned result.
4. Test invalid input or a failure condition, where applicable.
5. Record the observed output and any errors.





6. Observations

Record the following after completing the prescribed practical:

Item	Observation to record
Low-code application	Loan Application and Pre-screening Management
Extension requirement	The specific task selected from the manual
Pro-code component	Actual function or service used
Input	Sample input supplied during testing
Output	Actual result returned
Error handling	Observed behavior for invalid input or failure
Integration status	Whether the prescribed integration test succeeded

7. Advantages

1. Extends the functionality of low-code applications.
2. Supports complex processing requirements.
3. Allows integration with external systems.
4. Provides flexibility in implementing specialized logic.
5. Enables a hybrid development approach.

8. Result

The role of pro-code development in extending a low-code application was studied. The specified custom component and integration should be documented according to the actual steps performed and the results observed.

9. Conclusion

Pro-code extensions can complement low-code development by providing flexibility for advanced business requirements. Successful implementation requires appropriate architecture, secure integration, testing, error handling, and maintenance.